

SAC (Soft Actor-Critic)

先讲一个故事

你是一个练习投篮的运动员。经过几百次训练，你发现从左侧 45 度角切入投篮成功率最高，于是你开始只用这条路线。教练皱起眉头说：「你的左侧上篮确实稳，但你只有一招，对手摸清了规律，就会专门堵你。而且——万一右侧其实有条更好的路线，你永远不会发现。」

普通 RL 的做法：找到一条「最优」路线后死守，策略变成确定性的。

最大熵 RL 的做法：在保证高得分的前提下，尽量保持多种路线的可能性。随机性本身就是值得追求的——它带来探索、带来鲁棒性，也让你不那么容易被针对。

这就是最大熵 RL 与普通 RL 的分野：

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha H(\pi(\cdot|s_t))]$$

数学形式化

Shannon 熵定义（策略的随机性度量）：

$$H(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi(\cdot|s)} [\log \pi(a|s)]$$

- 均匀分布 $\rightarrow$ 熵最大；确定性策略 $\rightarrow$ 熵为 0；
- $\alpha > 0$ 为温度参数，控制「探索」与「利用」的权衡； $\alpha \rightarrow 0$ 退化为普通 RL。

翻译成人话：原来的目标是「只最大化奖励」，现在变成「最大化奖励 + α 倍的随机程度」。策略越随机，额外获得越多；策略越好，奖励项越高——两者同时被优化。

软贝尔曼方程

从硬到软

普通 Q 函数满足 Bellman 方程：

$$Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q(s', a') \right]$$

在最大熵框架下，策略不再取 max，而是对所有动作取期望，同时加上熵项。定义**软状态价值函数**：

$$V_{\text{soft}}(s) = \mathbb{E}_{a \sim \pi} [Q_{\text{soft}}(s, a) - \alpha \log \pi(a|s)]$$

对应的**软 Q 函数**（软贝尔曼方程）：

$$Q_{\text{soft}}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p} [V_{\text{soft}}(s')]$$

直觉： V_{soft} 比普通 V 多了一项 $-\alpha \log \pi$ 。在高熵（随机）状态， $-\alpha \log \pi$ 较大， V_{soft} 更高——算法主动奖励那些「不确定、可探索」的状态。

最优软策略

在软 Bellman 方程下，最优策略有解析形式：

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q_{\text{soft}}^*(s, a)\right)$$

这是一个 Boltzmann（能量）分布：Q 值越高的动作，被选中的概率越高；温度 α 越高，分布越均匀（更随机）； $\alpha \rightarrow 0$ ，退化为 $\arg \max$ （确定性贪心）。

SAC 的三个核心机制

机制一：Tanh 压缩高斯策略（重参数化采样）

连续动作空间中，策略输出一个高斯分布 $\mathcal{N}(\mu_\phi(s), \sigma_\phi^2(s))$ ，再用 tanh 压缩到有界区间。

$$a = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \varepsilon), \quad \varepsilon \sim \mathcal{N}(0, I)$$

重参数化的好处：将随机性 ε 从网络参数中「分离出来」，梯度可以直接对 ϕ 反向传播：

$$\nabla_\phi \mathbb{E}_a[f(a)] = \mathbb{E}_\varepsilon[\nabla_\phi f(\tanh(\mu_\phi + \sigma_\phi \odot \varepsilon))]$$

对数概率的 Tanh 修正（变量替换定理）：

$$\log \pi(a|s) = \log \mathcal{N}(u|\mu_\phi, \sigma_\phi^2) - \sum_d \log(1 - \tanh^2(u_d))$$

其中 $u = \mu_\phi + \sigma_\phi \odot \varepsilon$ 是压缩前的高斯样本。修正项 $-\sum_d \log(1 - \tanh^2(u_d))$ 恒非负：忽略它会低估动作概率（log-prob 偏小、熵偏大），且动作越靠近边界（ $|a_d| \rightarrow 1$ ）误差越大。

机制二：双 Q 网络（Clipped Double Q）

用单个 Q 网络计算目标值会导致**过高估计（overestimation bias）**：网络自举（bootstrapping）时倾向于选择被高估的 Q 值，误差不断累积。

SAC 维护两个独立的 Q 网络 $Q_{\theta_1}, Q_{\theta_2}$ ，计算目标时取最小值：

$$y = r + \gamma \left[\min_{i=1,2} Q_{\bar{\theta}_i}(s', \tilde{a}') - \alpha \log \pi(\tilde{a}'|s') \right], \quad \tilde{a}' \sim \pi(\cdot|s')$$

为什么取最小？两个网络的初始化与训练动态不同，估计误差**不完全相关**；取 min 相当于系统性偏好两者中更悲观的估计——宁可低估也不高估，阻断了过估计的正反馈。

各 Q 网络的损失：

$$L_Q(\theta_i) = \hat{\mathbb{E}} \left[(Q_{\theta_i}(s, a) - y)^2 \right]$$

机制三：软目标网络更新（Soft Target Update）

普通 DQN 的目标网络每隔固定步骤「硬拷贝」一次，容易引发训练抖动。SAC 用指数移动平均平滑更新：

$$\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i, \quad \tau \ll 1 \text{ (如 0.005)}$$

$\tau = 0.005$ 意味着目标网络每步只移动 0.5%，目标值变化极为平缓，稳定性大幅提升。

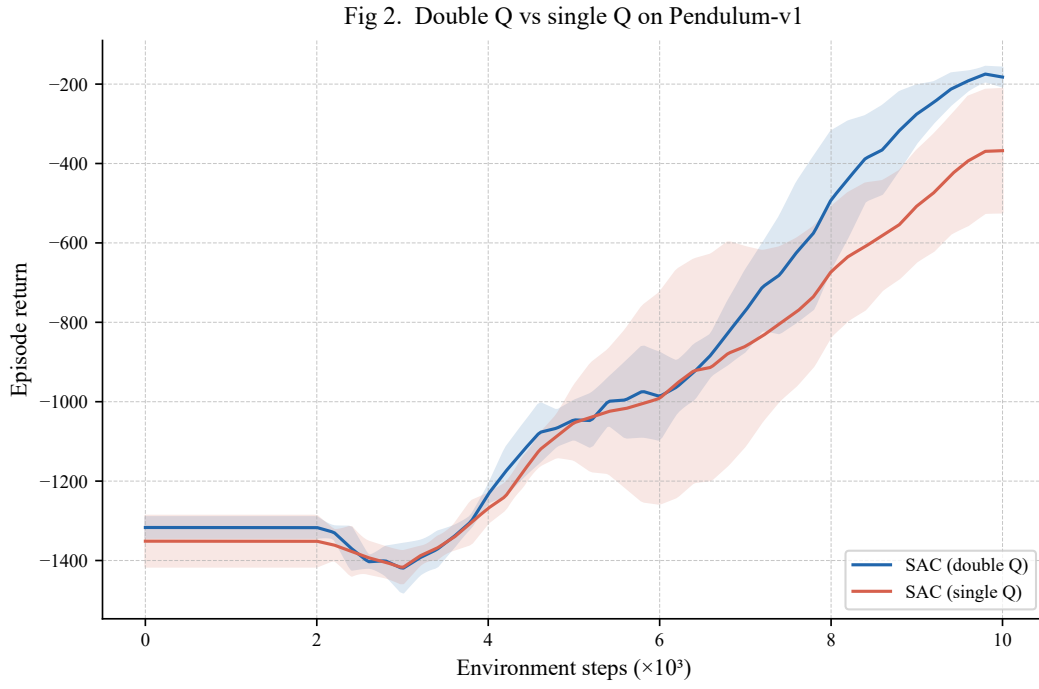


图 1: 双 Q vs 单 Q 消融 (Pendulum-v1, 3 个随机种子, 阴影为 $\pm 1\sigma$)。取 min 的双 Q 版本回报更高、更稳。

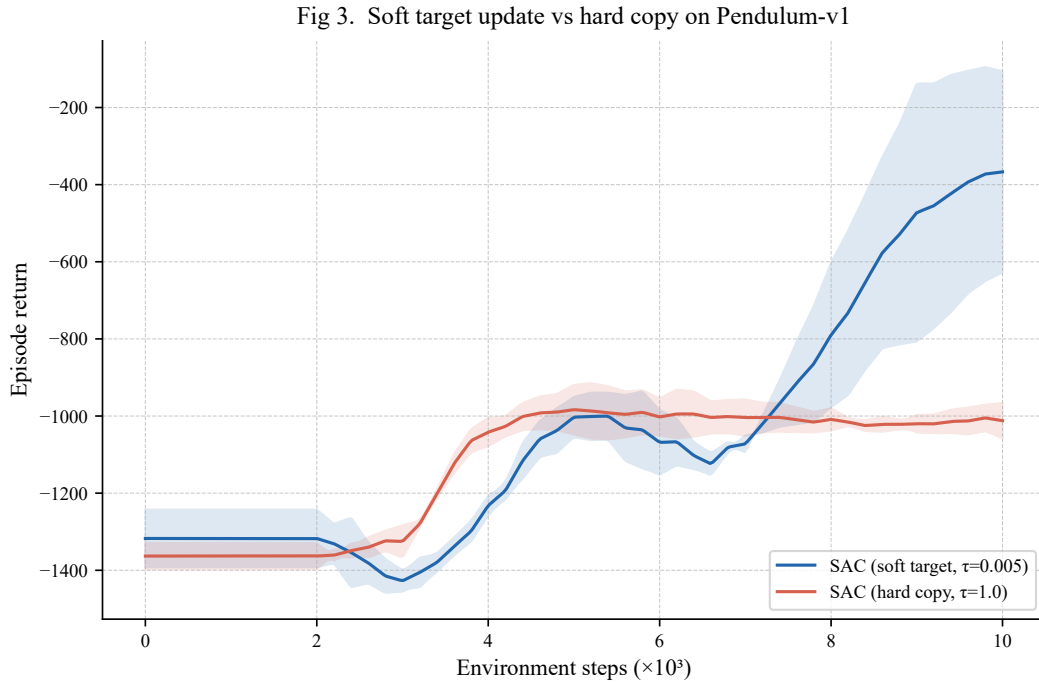


图 2: 软目标更新 ($\tau = 0.005$) vs 硬拷贝 ($\tau = 1.0$) 消融 (Pendulum-v1, 3 个随机种子, 阴影为 $\pm 1\sigma$)。软更新更平滑。

自动温度调节 (Automatic Entropy Tuning)

问题： α 怎么设？

α 决定探索强度：太大 $\rightarrow$ 策略太随机，不收敛；太小 $\rightarrow$ 退化为贪心，失去探索优势。手动调参代价高且脆弱。

SAC 用对偶梯度下降**自动学习** α ，目标是维持策略熵等于预设的目标熵 $\mathcal{H}_{\text{target}}$ ：

$$L(\alpha) = \mathbb{E}_{\tilde{a} \sim \pi} [-\alpha (\log \pi_{\phi}(\tilde{a}|s) + \mathcal{H}_{\text{target}})]$$

直觉：

- 若当前策略熵 $H(\pi) > \mathcal{H}_{\text{target}}$ (太随机) $\rightarrow$ 梯度让 α 下降，减弱探索；
- 若 $H(\pi) < \mathcal{H}_{\text{target}}$ (太确定) $\rightarrow$ 梯度让 α 上升，加强探索。

α 本质上是一个「拉格朗日乘子」，强制执行熵约束。

目标熵的设置

实践中常用启发式：

$$\mathcal{H}_{\text{target}} = -\dim(\mathcal{A})$$

对于 d 维连续动作空间，目标熵为 $-d$ (每维贡献 -1 纳特)。这等价于要求策略在每个维度的「有效随机性」约等于标准正态。

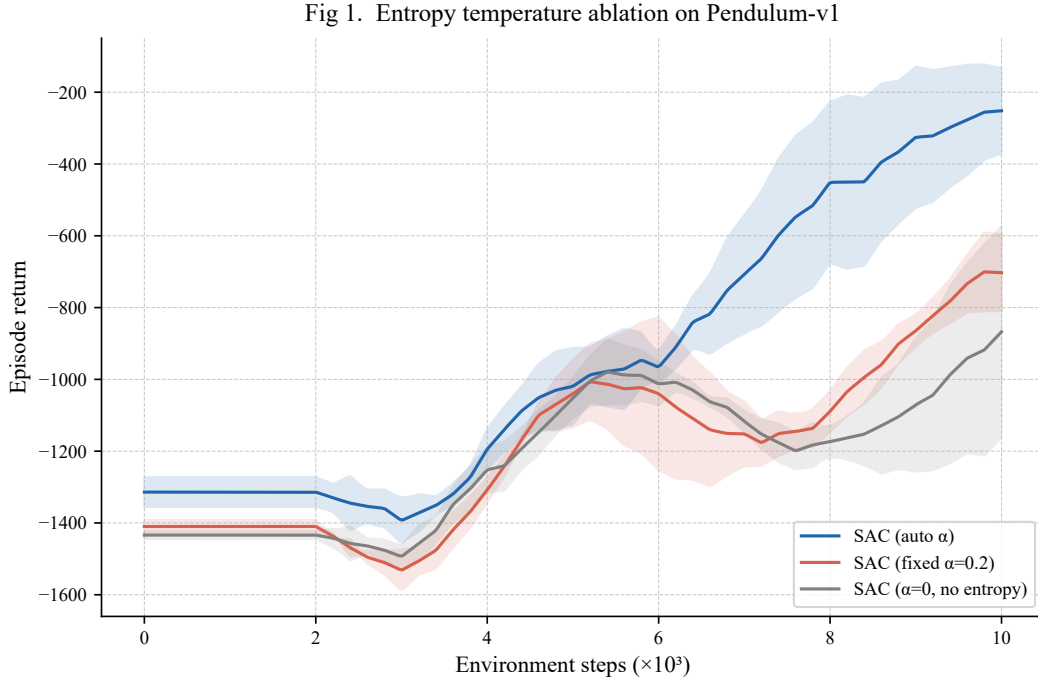


图 3: 温度消融：自动 α vs 固定 $\alpha = 0.2$ vs $\alpha = 0$ (Pendulum-v1, 3 个随机种子，阴影为 $\pm 1\sigma$)。自动调节通常更稳。

SAC 完整算法

三个并行优化目标

$$L_Q(\theta_i) = \hat{\mathbb{E}} \left[(Q_{\theta_i}(s, a) - y)^2 \right]$$

$$L_\pi(\phi) = \hat{\mathbb{E}}_{s, \tilde{a}} \left[\alpha \log \pi_\phi(\tilde{a}|s) - \min_i Q_{\theta_i}(s, \tilde{a}) \right]$$

$$L(\alpha) = \hat{\mathbb{E}}_{\tilde{a}} \left[-\alpha (\log \pi_\phi(\tilde{a}|s) + \mathcal{H}_{\text{target}}) \right]$$

Actor 损失的含义：最大化「Q 值 - α 倍的对数概率」= 最大化（奖励期望 + 熵）。

SAC 算法流程

1. **初始化**：Actor π_ϕ ，双 Critic $Q_{\theta_1}, Q_{\theta_2}$ ，目标网络 $Q_{\bar{\theta}_1}, Q_{\bar{\theta}_2}$ ，Replay Buffer $\mathcal{B}$ ， $\log \alpha$ ；
2. **收集数据**：用 π_ϕ 与环境交互，将 (s, a, r, s', d) 存入 $\mathcal{B}$ ；
3. **每步更新**（从 $\mathcal{B}$ 采样 mini-batch）：
 - (a) 计算软目标值 y （用目标网络 + 下一步动作 + 熵修正）；
 - (b) 更新双 Critic：最小化 $L_Q(\theta_1) + L_Q(\theta_2)$ ；
 - (c) 重采样 $\tilde{a} \sim \pi_\phi(\cdot|s)$ ，更新 Actor：最小化 $L_\pi(\phi)$ ；
 - (d) 更新温度：最小化 $L(\alpha)$ ；
 - (e) 软更新目标网络： $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i$ 。

SAC vs 其他深度 RL 方法

SAC vs PPO

方法	动作空间	样本效率	稳定性	离线数据
PPO	离散/连续	中 (on-policy)	高	难以复用
SAC	连续	高 (off-policy)	高	可复用，需保守化

为什么 SAC 比 PPO 更适合连续动作？

1. **Off-policy 样本效率**：SAC 使用 Replay Buffer，每条经验可被反复利用；PPO 是 on-policy，rollout 用完即丢；
2. **高维连续动作**：PPO 的离散化或 Gaussian 策略在高维空间方差大；SAC 的 Tanh-Gaussian + 重参数化梯度更稳定；
3. **自动探索调节**：SAC 的 α 自适应；PPO 的熵系数 c_2 需手动设定且固定；
4. **离线数据复用**：SAC 的 off-policy 结构可以复用历史与离线数据；PPO 的 on-policy 结构很难适配离线数据。注意：复用离线数据不等于可以直接离线训练——裸 SAC 在纯离线数据上同样会因外推误差塌缩，需要 CQL/IQL 式的保守化改造（见下一章）。

本章在 RL 体系中的位置

1. **TD 路线** → Q-Learning → DQN (离散动作, 值函数方法);
2. **MC 路线** → REINFORCE → 策略梯度 (on-policy, 高方差);
3. **Actor-Critic** → GAE → PPO (在线策略, clip 约束);
4. **最大熵 RL** → SAC (本章: 连续动作, off-policy, 自动熵调节)。

SAC 是连续控制任务的默认基线: 样本高效、训练稳定、自动调节探索强度, 是工业界和学术界连续控制任务的首选算法。

参考资料

1. Haarnoja et al. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*.
2. Haarnoja et al. (2018). Soft Actor-Critic Algorithms and Applications. *arXiv:1812.05905*.
3. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapter 13.